High-Traffic Event Booking, Last-Minute Ticket Exchange & Event Ride-Pooling System Using Service-Oriented Architecture

SOA Project Review-2

Under the Guidance of

Navyatha Ravi
XXXXXX Professor,
Department of Computer Science and Engineering,
KLH CSE Bowrampet Campus

Team Members
Meghna – 2420030401
Thanusree - 2420030442

Outline of Project Work



Problem Statement

1 High Traffic

- Popular events draw thousands of users at the same moment
- Risk of overload, slow responses and double booking
- Needs rate limiting, queuing and seat concurrency control

2 Unused Tickets

- Tickets go unused when buyers cannot attend
- Cancellation leaves seats empty
- No safe way to transfer ownership before the event

3 Event Transportation

- Thousands travel to one venue at similar times
- Congestion and high individual travel cost
- No link between ticket holders going to the same event

Existing platforms stop at ticket purchase. There is no integrated way to withstand booking surges, recover unused tickets, or coordinate travel to the venue, all centred on one event.

Objectives

1. Build an SOA-based event booking platform with the standard booking workflow
2. Prevent double booking through seat locking and concurrency control
3. Handle traffic surges with API Gateway, rate limiting and RabbitMQ queues
4. Provide last-minute ticket exchange with configurable cutoff, refund and pricing rules
5. Enable event-based ride pooling using rule-based matching
6. Ensure data isolation, fault isolation and independent scaling of services

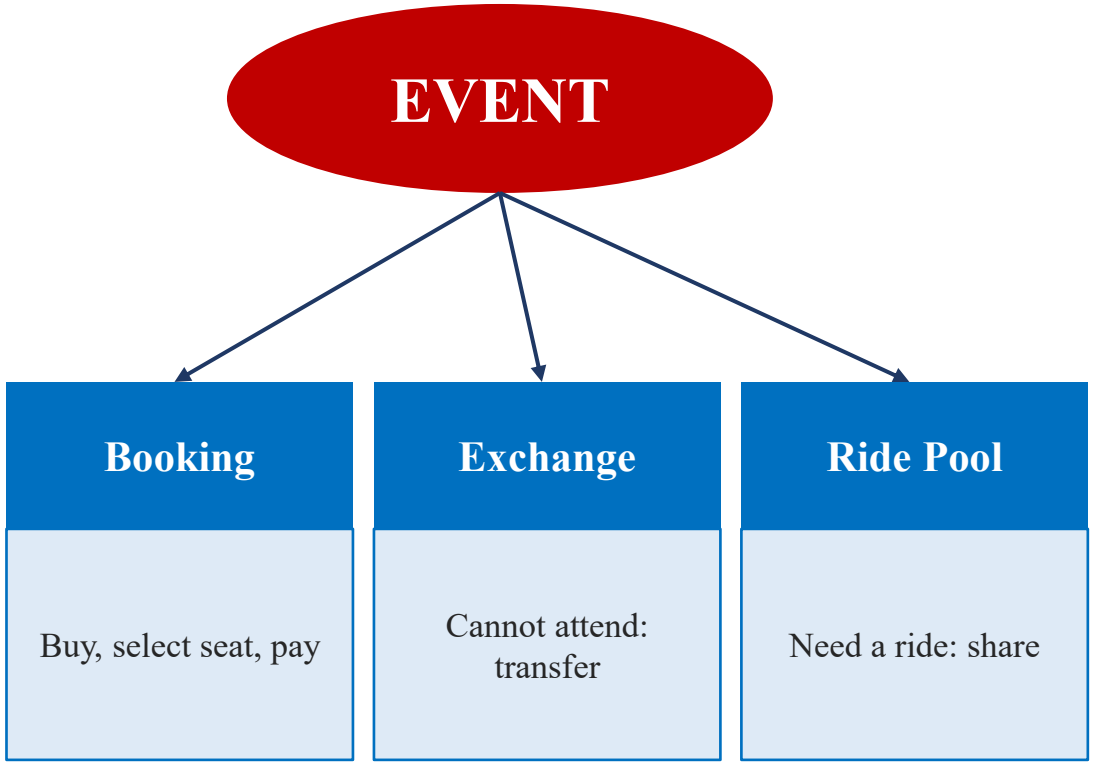
Literature Survey

Authors	Title of the Paper	Methodology	Inferences	Limitation/Future work
Dragoni et al. [1]	Microservices: Yesterday, Today, and Tomorrow, Springer, 2017	Survey of microservice origins, principles and evolution from SOA	Independent deployment, scaling and fault isolation compared with monoliths	No booking or ticketing case study; design left generic
Garcia-Molina & Salem [2]	Sagas, ACM SIGMOD, 1987	Splits long transactions into sub-transactions with compensating actions	Keeps data consistent across services without long-held locks	Database-centric; must be adapted to REST and message-driven services
Alonso-Mora et al. [3]	On-demand high-capacity ride-sharing via dynamic trip-vehicle assignment, PNAS, 2017	Graph-based dynamic assignment of vehicles to ride requests	Shared rides serve demand with fewer vehicles	City-scale taxi model; not tied to venues or ticket holders
Furuhata et al. [4]	Ridesharing: The State-of-the-Art and Future Directions, Transportation Research Part B, 2013	Classifies ridesharing systems and matching challenges	Pickup proximity and time-window matching are core requirements	No event-specific demand or ticketing integration

Problem Gap Identification

Area	Gap in existing platforms	Our approach
High Traffic	Systems slow down or fail under sudden surges; seats can be double booked	Gateway rate limiting, RabbitMQ waiting queue and Seat Service locks
Unused Tickets	Cancellation wastes seats; resale is informal and unsafe	Exchange Service: eligibility check, ownership transfer, refund or credit
Event Transport	Travel is handled separately from the ticket	Ride Pool Service matches same-event attendees by pickup, time and capacity

Innovation, Creativity, and Novelty



- The event is the common entity connecting booking, ownership and transport
- Last-minute exchange turns unused seats into safely resold tickets
- Event-based ride pooling reuses ticket data to match attendees
- Traffic-aware SOA (gateway, queue, seat locks) keeps booking consistent
- Extends existing platforms such as BookMyShow and District rather than replacing them

One event links ticketing, ownership and transport

Feasibility and Project Plan

Technical

- Mature stack: Spring Boot, MySQL, Redis, RabbitMQ
- Rule-based matching, no AI model needed

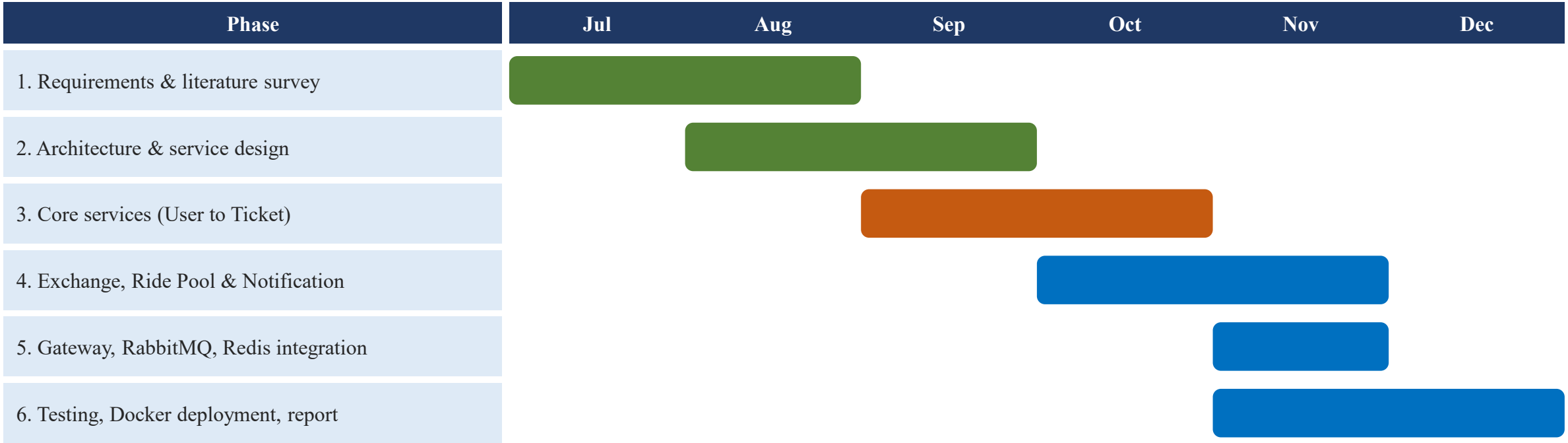
Operational

- Docker runs all services locally
- Mock payment and simulated notifications

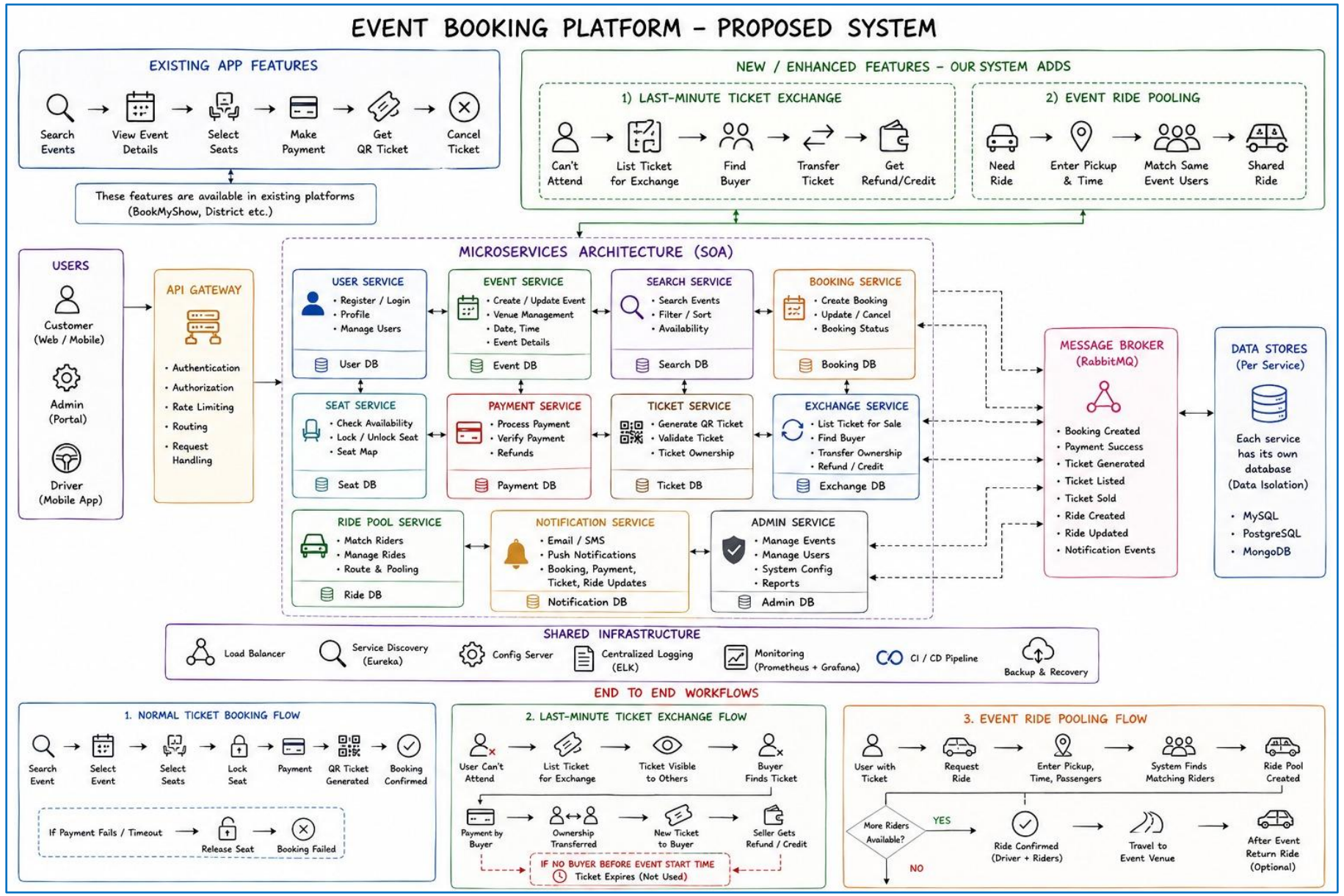
Economic

- Fully open-source tools
- No paid gateway or SMS services

Project Timeline (Jul-Dec 2026) ■ Completed ■ In progress ■ Planned



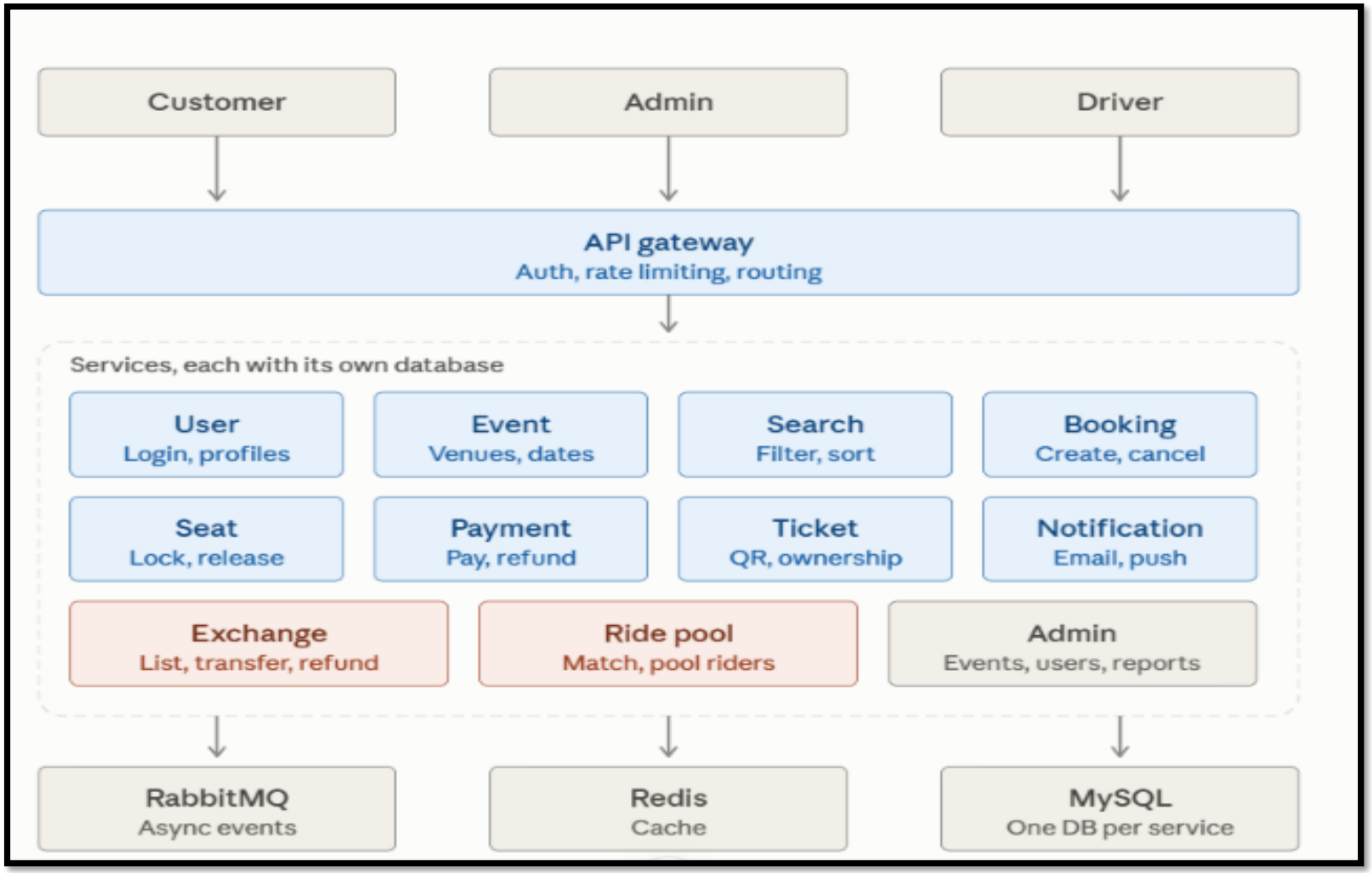
Project Design



Design Highlights

- API Gateway: single entry point with authentication, authorization and rate limiting
- 11 independent services, each with its own database
- REST for immediate calls (Booking to Seat, Payment)
- RabbitMQ events for notifications, tickets and rides
- Redis caches event details and seat availability
- Eureka discovery, load balancer, ELK logging, Prometheus and Grafana monitoring

System Architecture



Methodology Adopted



Development Approach

- Agile, iterative sprints: core booking first, then exchange, then ride pool
- SOA decomposition with one database per service
- Contract-first REST APIs, tested with Postman
- Event-driven messaging with RabbitMQ to decouple services
- Tools: Spring Boot, Spring Cloud Gateway, Eureka, MySQL, Redis, React, Docker, JUnit

Core Rules and Algorithms

- Seat lock with timeout: AVAILABLE, LOCKED, BOOKED
- Exchange: configurable cutoff, refund policy and pricing rules
- Ride matching: same event and destination, pickup distance and arrival-time thresholds, vehicle capacity
- Rule-based only; no AI model required
- Mock payment gateway and simulated notifications for the academic scope

Technical Soundness

Seat Consistency

- Seat states AVAILABLE, LOCKED, BOOKED
- Lock expires on timeout or payment failure
- Atomic lock prevents double booking

Cross-Service Consistency

- Saga-style compensation: failed payment releases the seat
- Idempotent payment and ownership-transfer calls
- Exchange keeps one valid ticket owner

High-Traffic Handling

- Gateway rate limiting and waiting queue
- RabbitMQ smooths load on Booking DB
- Redis cache; services scale independently

Fault Isolation and Validation

- Timeouts, retries and dead-letter queues
- A failing Notification or Ride service does not block booking
- Testing: JUnit, Postman, planned load tests